

SENG 8323 Software Modeling Design

Lecturer: Placide RWEGO MUCYO

1. Course Description

- **Software Modeling and Design** is a software engineering course that introduces students to the **principles, techniques, and tools** used **to analyze, model, design, and document software systems**. The course emphasizes the use of Unified Modeling Language (**UML**) and **object-oriented design principles** to transform software requirements into well-structured and maintainable system designs.
- Students learn how to represent system **structure, behavior, interactions, and architecture** through different models and diagrams.
- The course also covers software architecture, design patterns, design principles, and the transition from analysis models to implementation.

2. Course Objectives

students should be able to:

- 1.Explain fundamental concepts of software modeling and design.
- 2.Develop UML models for software systems.
- 3.Create structural and behavioral models.
4. Apply object-oriented design principles.

students should be able to:

- 5.Design software architectures appropriate to system requirements.
- 6.Apply common software design patterns.
- 7.Evaluate alternative software designs.
- 8.Produce clear and consistent software design documentation.
- 9.Use modeling tools to develop and maintain software models.

Topic 1: Introduction to Software Modeling Design

Software modeling is the process of creating a simplified visual or conceptual representation of a software system to help developers, analysts, designers, and stakeholders understand how the system will work before it is implemented.

A software model represents important aspects of a system, such as:

Structure – what components, classes, objects, and databases the system contains.

Behavior – how the system responds to events and performs activities.

Interactions – how users and system components communicate.

Data – how information is organized and related.

Architecture – how major software components are organized and connected.

Why Software Modeling is Important?

- Software modeling helps to:

1. **Understand complex systems** – simplifies complicated software concepts.
2. **Communicate ideas** – provides a common language between developers, users, and stakeholders.
3. **Identify problems early** – design errors can be detected before coding.
4. **Guide implementation** – models provide a blueprint for programmers.
5. **Document the system** – models serve as technical documentation.
6. **Improve software quality** – encourages organized and systematic design.
7. **Support maintenance** – developers can understand the existing system more easily.

software design

- **Software design** is the process of **planning and defining the structure, components, interfaces, data, and interactions of a software system** before the system is implemented through programming.
- In simple terms, software design answers the question:
“How will the software system be built to satisfy the identified requirements?”
- **Simply** a software design is the process of transforming software requirements into a detailed blueprint that describes how a software system will be structured and how its components will work together.

Key Elements of Software Design

- Software design typically involves determining:
 1. **System architecture** – the overall organization of the software.
 2. **Components and modules** – the major parts of the system.
 3. **Classes and objects** – the entities that make up an object-oriented system.
 4. **Data structures** – how data will be stored and organized.
 5. **Interfaces** – how users and software components interact.
 6. **Algorithms and processes** – how specific operations will be performed.
 7. **Communication between components** – how different parts of the system exchange information.
 8. **Security and performance considerations** – how the design will meet non-functional requirements.

Importance of Software Design

Software design is important because it:

- Provides a **blueprint for implementation**.
- Reduces development errors and rework.
- Makes complex systems easier to understand.
- Improves **maintainability and scalability**.
- Promotes software **reuse**.
- Helps developers identify design problems before coding.
- Improves communication among development team members.
- Ensures that the implementation addresses the system requirements.

Software Modeling vs. Software Design

Software Modeling

Represents the system using models and diagrams

Focuses on understanding and communicating the system

Uses UML diagrams extensively

Answers **“What does the system look like?”**

Software Design

Defines how the system will actually be structured and built

Focuses on creating an implementation blueprint

Uses architecture, components, interfaces, classes, patterns, etc.

Answers **“How will the system be built?”**

Importance of modeling in software development

- Software modeling is important because it provides a simplified representation of a software system before and during implementation.
- **Models** help developers and stakeholders understand the system, communicate requirements, identify problems, and guide the development process.

Importance of modeling in software development

1. Simplifies Complex Systems

Software systems can contain thousands of components and interactions.

Modeling reduces this complexity by representing only the most important aspects of the system.

Example: A class diagram can show the major classes and relationships without showing all the programming code.

2. Improves Communication

Models provide a common visual language for developers, analysts, designers, customers, and other stakeholders.

For example, a UML use case diagram can help a customer understand what users will be able to do in the system.

Importance of modeling in software development

3. Helps Understand Requirements

Modeling helps transform unclear or textual requirements into visual representations.

Example:

Requirement: “A customer should be able to **book a bus ticket**.”

This can be represented **using a use case** diagram showing:

Customer → Book Ticket

4. Detects Errors Early

Creating models before implementation makes it possible to identify missing requirements, incorrect relationships, and inconsistencies before significant coding effort is spent.

Importance of modeling in software development

5. Provides a Blueprint for Development

Models act as **a blueprint** for programmers.

They show how components, classes, databases, and processes should be organized.

**Requirements → Models → Design
→ Implementation**

6. Supports Better System Design

Modeling allows developers to compare different design alternatives before choosing an implementation approach.

For example, developers can evaluate whether a system should use:

Layered architecture

Client-server architecture

MVC architecture

Microservices architecture

Importance of modeling in software development

7. Reduces Development Cost

Finding and correcting problems during modeling is generally **less expensive** than discovering them after the system has been implemented.

8. Improves Software Quality

Well-developed models encourage important design qualities such as:

Modularity

Reusability

Maintainability

Scalability

Low coupling

High cohesion

Importance of modeling in software development

9. Supports Documentation

Models provide technical documentation that **explains how** the system is **structured and behaves**. This is particularly useful when new developers join a project or when the system needs maintenance.

10. Supports System Maintenance

After deployment, developers can use existing models to understand the system and determine the potential effects of proposed changes.

Importance of modeling in software development

11. Facilitates Teamwork

In large software projects, different developers may work on different parts of the system.

Models help team members understand how their components fit into the overall system.

12. Supports Reuse

Models can identify reusable classes, components, interfaces, and design patterns that can be used in other systems.

Software Development Life Cycle and Design Activities

- **1. Software Development Life Cycle (SDLC)**
- Software Development Life Cycle, SDLC for short, is a well-defined, structured sequence of stages in software engineering to develop the intended software product.
- The Software Development Life Cycle (SDLC) is a systematic process used to plan, develop, test, deploy, and maintain software systems.
- It provides a structured approach for transforming a software idea or business problem into a working software product.

1. Software Development Life Cycle (SDLC)

Main Stages of the SDLC

**Planning → Requirements
Analysis → Design →
Implementation → Testing →
Deployment → Maintenance**

1. Planning

This is the initial stage where the organization identifies:

- The problem to be solved
- Project goals and objectives
- Project scope
- Required resources
- Estimated cost
- Project schedule
- Technical feasibility
- Risks and constraints

2. Requirements Analysis

At this stage, developers and analysts determine what the **system should do**.

Activities include:

- Identifying stakeholders**
- Eliciting requirements**
- Analyzing requirements**
- Documenting requirements**
- Validating requirements**
- Prioritizing requirements**

3. Software Design

The design stage determines **how the system will satisfy the requirements**.

Designers develop a blueprint of the system, including:

- System architecture**
- Software components**
- Classes and objects**
- Database structure**
- User interfaces**
- System interactions**
- Security mechanisms**
- Communication between components**

4. Implementation

In this stage, programmers translate the design into **source code**.

Activities include:

- Writing program code**

- Creating databases**

- Implementing interfaces**

- Integrating components**

- Performing code reviews**

- Managing source code**

5. Testing

Testing determines whether the software works correctly and satisfies its requirements.

Common testing activities include:

- Unit testing**

- Integration testing**

- System testing**

- Acceptance testing**

- Performance testing**

- Security testing**

- Usability testing**

The objective is to identify and correct defects before the system is released.

6. Deployment

The completed software is installed and made available to its intended users.

Activities may include:

- Installing the system**

- Configuring servers**

- Migrating data**

- Training users**

- Configuring networks**

- Releasing the software**

7. Maintenance

After deployment, the system requires continuous maintenance.

Maintenance may involve:

- Correcting errors**

- Improving performance**

- Adding new features**

- Updating security**

- Adapting the system to new environments**

2. Design Activities in the SDLC

- **Design activities** are the activities performed to transform software requirements into a technical blueprint that programmers can use to implement the system.

The major design activities include:

A. Architectural Design

B. Data Design

C. Interface Design

D. Component-Level Design

E. Interaction Design

F. Behavioral Design

G. Security Design

H. Design Review and Validation

2. Design Activities in the SDLC

A. Architectural Design

Defines the overall **structure of the software** system. It determines:

- Major system components**
- Relationships between components**
- Architectural style**
- Communication mechanisms**
- Technology choices**

Examples:

- Layered architecture**
- Client-server architecture**
- MVC**

B. Data Design

Determines how system data will be organized, stored, accessed, and managed.

Activities include:

- Identifying data entities**
- Defining attributes**
- Designing databases**
- Defining relationships**
- Selecting data structures**

Defining primary and foreign keys

Example: A bus booking system may have:

Passenger → Booking → Bus → Route → Payment

2. Design Activities in the SDLC

E. Interaction Design

Defines how different components or objects communicate with each other.

For example, when a passenger books a ticket:

Passenger → Booking System → Seat Service → Payment Service → Ticket Service

A sequence diagram can represent this interaction.

F. Behavioral Design

Describes how the system behaves in response to events and user actions.

Models commonly used include:

Activity diagrams

State machine diagrams

Sequence diagrams

2. Design Activities in the SDLC

C. Interface Design

Defines how users and software components will interact. It includes:

User interface design

Navigation

Input forms

Menus

Buttons

Messages

APIs

Interfaces between components

D. Component-Level Design

Defines the internal structure and responsibilities of individual software components.

Designers determine:

Classes

Methods

Attributes

Interfaces

Responsibilities

Relationships between classes

A class diagram is commonly used for this activity

2. Design Activities in the SDLC

G. Security Design

Identifies mechanisms needed to protect the system and its data.

Examples include:

Authentication

Authorization

Encryption

Access control

Secure data storage

Audit logging

H. Design Review and Validation

The proposed design is reviewed to determine whether it:

Satisfies requirements

Is complete

Is consistent

Is feasible

Is secure

Is maintainable

Has acceptable performance

3. Relationship Between SDLC and Design

The design stage connects requirements with implementation:

- Requirements



- Software Design



- UML Models & Architecture



- Implementation/Coding



- Testing



- Deployment

Example: Online Bus Ticket Booking System

SDLC Stage	Example Activity
Planning	Decide to develop an online booking system
Requirements	Identify passenger and administrator requirements
Design	Create architecture, UML diagrams, database and interface designs
Implementation	Write the booking system code
Testing	Test booking, payment, cancellation, etc.
Deployment	Release the system to passengers
Maintenance	Fix problems and add new features

- The **SDLC** provides the **overall process for developing software**, while design activities focus **specifically on defining how the software will be structured and implemented** to satisfy its requirements.

Analysis versus Design in Software Development

- Software analysis and software design are closely related activities, but they have different purposes.

1. Software Analysis

- Software analysis is the process of examining the problem domain and determining what the software system must do to satisfy users and business requirements.
- It focuses on understanding the problem, users, requirements, data, and system behavior without deciding exactly how the system will be implemented.

Main question:

- “What does the system need to do?”

Analysis versus Design in Software Development

- **2. Software Design**

- Software design is the process of transforming the analyzed requirements into a technical blueprint that defines how the system will be built.
- It focuses on the architecture, components, classes, interfaces, database, algorithms, and interactions of the system.
- **Main question:**
How will the system do it?

Analysis vs. Design

Aspect	Software Analysis	Software Design
Main focus	Understanding the problem	Creating the solution
Main question	What?	How?
Purpose	Identify and understand requirements	Define how requirements will be implemented
Concern	Problem domain	Solution/implementation domain
Activities	Requirements elicitation, analysis and validation	Architecture, database, interface and component design
Output	Analysis model and requirements specification	Design model and technical blueprint

Analysis vs. Design

Technology	Generally technology-independent	Often considers technologies and platforms
UML examples	Use case and domain models	Class, sequence, component and deployment diagrams
Main participants	Customers, stakeholders, analysts	Software architects, designers and developers
Timing	Generally before detailed design	Follows requirements analysis

Meaning of a Software Model

- A software model is a **simplified representation** of a software system that describes its structure, behavior, data, or interactions.
- Models can be represented using **diagrams, text, mathematical notation, tables, or other formal representations.**

Examples of Software Models

- Common software models include:
- **Use case model** – represents users and their interactions with the system.
- **Class model** – represents classes, attributes, methods, and relationships.
- **Sequence model** – represents interactions between objects over time.
- **Activity model** – represents workflows and processes.
- **State model** – represents how an object changes from one state to another.
- **Component model** – represents software components and their dependencies.
- **Deployment model** – represents how software components are deployed on hardware.

Meaning of Abstraction

- Abstraction is the process of hiding unnecessary details and focusing only on the essential characteristics of a system or object.

In other words:

- Abstraction allows us to focus on what is important without being concerned with all the implementation details.
- In software development, **abstraction allows** developers to work with complex systems at different levels of detail.

For example, a user may see:

“Make Payment”

- while internally the system performs:

- **Validate customer**



- **Check account**



- **Verify payment details**



- **Process transaction**



- **Update database**



- **Generate receipt**

- The user **does not need to understand these internal operations.**

Levels of Abstraction

Software systems can be represented at different levels.

- **Level 1: High-Level Abstraction**

Shows the system from a broad perspective.

- **Example:**

Online Bus Booking System

|

|

|

|

Booking Payment Reporting

- It focuses on major system components.

Levels of Abstraction

Level 2: Intermediate Abstraction

Shows more detail about individual components.

Booking System

|

- +-- Search Bus
- +-- Select Seat
- +-- Create Booking
- +-- Cancel Booking

Levels of Abstraction

- **Level 3: Low-Level Abstraction**

Shows implementation details such as:

Classes

Methods

Variables

Algorithms

Database tables

Programming language structures

Importance of Models and Abstractions

They are important because they:

1. **Reduce complexity** – simplify complicated systems.
2. **Improve understanding** – make systems easier to understand.
3. **Improve communication** – provide a common language for stakeholders.
4. **Support problem solving** – allow developers to examine a system before implementation.
5. **Identify errors early** – inconsistencies can be detected in models.
6. **Support software design** – models provide a foundation for designing the system.
7. **Improve documentation** – models describe how the system works.
8. **Support maintenance** – developers can understand existing systems more easily.
9. **Promote reuse** – common structures and components can be identified.
10. **Manage complexity** – different levels of abstraction allow developers to focus on specific details.

Model vs. Abstraction

Model

A representation of a system

Shows selected aspects of a system

Can be a diagram, table, or textual description

Used to communicate and analyze a system

Example: UML class diagram

Abstraction

A process of simplifying a system

Determines which aspects are important

Hides unnecessary details

Used to manage complexity

Example: showing Book() without showing its internal code

Principles of Good Software Design

- **Good software design** is the process of creating a software structure that is correct, understandable, maintainable, reusable, secure, scalable, and efficient.
- A good design should satisfy current requirements while making it easier to modify and extend the system in the future.

Principles of Good Software Design

1. Simplicity

A good design should be as simple as possible while still meeting the requirements.

“Keep it simple.”

Avoid unnecessary components, complicated logic, and excessive dependencies.

Example: Use a simple method for calculating a ticket price instead of creating several unnecessary classes.

2. Modularity

Modularity means dividing a large system into smaller, independent modules or components.

For example, an online bus booking system could contain:

Online Bus Booking System



Each module performs a specific responsibility.

Benefit: Modules can be developed, tested, modified, and maintained independently.

Principles of Good Software Design

3. Separation of Concerns

Each part of the system should focus on a specific concern or responsibility.

For example:

- User interface handles presentation.

- Booking service handles bookings.

- Payment service handles payments.

- Database layer handles data storage.

This prevents unrelated responsibilities from being mixed together.

4. High Cohesion

Cohesion refers to how closely related the responsibilities within a module are.

A module should contain functions that are strongly related to one another.

Example

A PaymentService should handle:

- Processing payments

- Validating payment information

- Confirming payment

It should not also be responsible for generating bus schedules.

Good design aims for high cohesion.

Principles of Good Software Design

5. Low Coupling

Coupling refers to the degree of dependency between software components.

A good design minimizes unnecessary dependencies between modules.

For example:

Booking Module → Payment Interface
is preferable to making the booking module directly depend on the internal implementation of a specific payment provider.

6. Encapsulation

Encapsulation means keeping an object's data and implementation details protected and providing controlled access through methods or interfaces.

For example:

BankAccount

- balance

+ deposit()

+ withdraw()

+ getBalance()

The balance should not normally be directly modified by external objects.

Principles of Good Software Design

7. Abstraction

Abstraction means focusing on the important characteristics of an object or system while hiding unnecessary implementation details.

For example, a user selects:

“Pay Now”

without needing to understand the internal processes used by the payment system.

8. Reusability

Software components should be designed so that they can be reused in different parts of the system or in other systems.

Examples include:

Reusable classes

Libraries

APIs

Components

Services

Design patterns

Reusability reduces development time and duplication.

Principles of Good Software Design

9. Maintainability

A good design should make the software easy to understand, modify, test, and repair.

Maintainable software usually has:

- Clear structure**

- Meaningful names**

- Modular components**

- Good documentation**

- Low coupling**

- High cohesion**

10. Scalability

A system should be designed so that it can handle increasing numbers of users, transactions, or data without major redesign.

For example, an online booking system should be capable of growing from:

1,000 users → 10,000 users → 1,000,000 users

while maintaining acceptable performance.

Principles of Good Software Design

11. Flexibility

A good design should accommodate changes in requirements with minimal modification.

For example, a payment system should be designed so that adding:

Mobile money

Credit cards

Bank transfers

does not require rewriting the entire booking system.

12. Reusability and Avoiding

A good design follows the principle:

DRY – Don't Repeat Yourself.

The same business logic should not be unnecessarily duplicated in multiple places. Instead, common functionality should be placed in a reusable component or method.

Principles of Good Software Design

13. Design for Change

Software requirements often change. Good designs anticipate reasonable future changes.

For example:

Payment Interface

|

| | |

Mobile Card Bank

Money Payment Transfer

Using an interface makes it easier to add new payment methods later.

14. Testability

Software components should be designed so they can be **easily tested independently**.

Modular design, low coupling, and clear interfaces improve testability.

For example, the payment service should be testable without requiring the entire bus booking system to run.

Principles of Good Software Design

15. Security

Security should be considered during design rather than added after implementation.

Important considerations include:

- Authentication**

- Authorization**

- Data encryption**

- Input validation**

- Secure communication**

- Access control**

- Protection of sensitive information**

16. Efficiency

A good design should use computing resources effectively.

Important considerations include:

- Processing time

- Memory usage

- Database performance

- Network usage

- Response time

However, optimization should not introduce unnecessary complexity.

SOLID Design Principles

Principle	Meaning
S - Single Responsibility	A class should have one main responsibility
O - Open/Closed	Software should be open for extension but closed for unnecessary modification
L - Liskov Substitution	Subclasses should be usable wherever their base class is expected
I - Interface Segregation	Prefer small, specific interfaces rather than large general interfaces
D - Dependency Inversion	Depend on abstractions rather than concrete implementations

The major characteristics of a good software model are:

- 1. Correctness**
- 2. Completeness**
- 3. Consistency**
- 4. Clarity**
- 5. Simplicity**
- 6. Appropriate abstraction**
- 7. Modularity**
- 8. Traceability**
- 9. Consistent level of detail**
- 10. Verifiability**
- 11. Modifiability**
- 12. Reusability**

.

END